

Experiment 04:

Laboratory Exercise:

Code:

```
src > TS index.ts > ...
1  class Person {
2      public id: number
3      private name: string
4      protected age: number
5
6      constructor(id: number, name: string, age: number) {
7          this.id = id
8          this.name = name
9          this.age = age
10     }
11
12     changeName(newName: string): void {
13         this.name = newName
14         console.log(`\n[PRIVATE] Name changed successfully`)
15         console.log(`Updated name: ${this.name}`)
16     }
17
18     register(): string {
19         return `${this.name} has registered`
20     }
21
22     displayDetails(): void {
23         console.log(`ID: ${this.id}`)
24         console.log(`Name: ${this.name}`)
25         console.log(`Age: ${this.age}`)
26     }
27 }
28
```

```
29 // Inheritance
30 class Employee extends Person {
31     public company: string
32
33     constructor(
34         id: number,
35         name: string,
36         age: number,
37         company: string
38     ) {
39         super(id, name, age)
40         this.company = company
41     }
42
43     // Method Overriding
44     register(): string {
45         return `Employee has registered successfully`
46     }
47 }
```

```
48 // Protected Variable
49 displayCompany(): void {
50     console.log(`ID: ${this.id}`)
51     console.log(`Company: ${this.company}`)
52     console.log(`Age: ${this.age}`)
53 }
54
55 // Method Overloading
56 displayEmployee(name: string): void
57 displayEmployee(name: string, company: string): void
58
59 displayEmployee(name: string, company?: string): void {
60     if (company) {
61         console.log(`Employee: ${name}`)
62         console.log(`Company: ${company}`)
63     } else {
64         console.log(`Employee: ${name}`)
65     }
66 }
67 }
68
69 // Public Variable
70 console.log("\n----- PUBLIC VARIABLE ----- \n")
71
72 const steve = new Person(1, "Steve Rogers", 46)
73
74 console.log(`ID is accessible: ${steve.id}`)
75
76 // Private Variable
77 console.log("\n----- PRIVATE VARIABLE ----- \n")
78
79 console.log(steve.register())
80
81 steve.changeName("Jhonny Storm")
82
```

```
83 // Protected Variable
84 console.log("\n----- PROTECTED VARIABLE ----- \n")
85
86 const matt = new Employee(
87     2,
88     "Matthew Murdock",
89     30,
90     "Hells Kitchen"
91 )
92
93 matt.displayCompany()
94
95 // Method Overriding
96 console.log("\n----- METHOD OVERRIDING ----- \n")
97
98 console.log(matt.register())
99
100 // Method Overloading
101 console.log("\n----- METHOD OVERLOADING ----- \n")
102
103 matt.displayEmployee("Bruce Wayne")
```

```
105  matt.displayEmployee(  
106      "Bruce Wayne",  
107      "Gotham"  
108  )  
109  
110  // Display Person Details  
111  console.log("\n----- PERSON DETAILS ----- \n")  
112  
113  steve.displayDetails()
```

Output:

```
C:\Exp_4_Shivam>npx tsc  
  
C:\Exp_4_Shivam>node dist/index.js  
  
----- PUBLIC VARIABLE -----  
  
ID is accessible: 1  
  
----- PRIVATE VARIABLE -----  
  
Steve Rogers has registered  
  
[PRIVATE] Name changed successfully  
Updated name: Jhonny Storm  
  
----- PROTECTED VARIABLE -----  
  
ID: 2  
Company: Hells Kitchen  
Age: 30  
  
----- METHOD OVERRIDING -----  
  
Employee has registered successfully  
  
----- METHOD OVERLOADING -----  
  
Employee: Bruce Wayne  
Employee: Bruce Wayne  
Company: Gotham  
  
----- PERSON DETAILS -----  
  
ID: 1  
Name: Jhonny Storm  
Age: 46  
  
C:\Exp_4_Shivam>
```

Post-Experiments Exercise:

- 1) hover over a, b, c, d and notice how and what typescript infers data types of all your variables.

Output:

```
let a: number
let a = 1 + 2
let b = a + 3
let c = {
  apple: a,
  banana: b
}
let d = c.apple * 4
```

```
let b: number
let b = a + 3
let c = {
  apple: a,
  banana: b
}
let d = c.apple * 4
```

```
let c: {
  apple: number;
  banana: number;
}
let c = {
  apple: a,
  banana: b
}
let d = c.apple * 4
```

```
let a = 1 + 2
let b = a + 3
let d: number
let d = c.apple * 4
```

'd' is declared but its value is never read. ts(6133)

Quick Fix... (Ctrl+.)

Variable	Inferred type	Value
a	number	3
b	number	6
c	{ apple: number; banana: number }	object
d	number	12

Explanation:

TypeScript automatically identifies the data type from the values and operations. You don't have to write **: number** yourself.

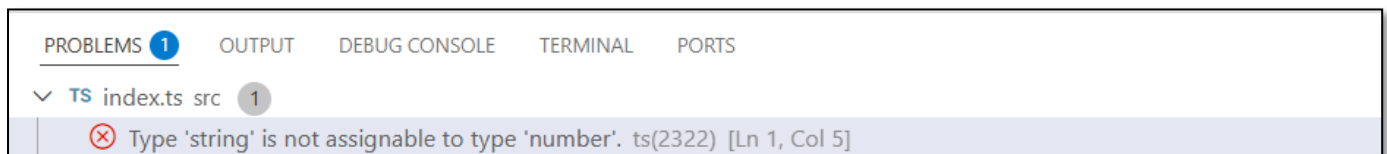
2) Play around the code and see if:

a) Get TypeScript to show a red squiggly when you do something invalid (we call this “throwing a Type Error”).

Code:

```
let a: number = "Hello"  
let b = a + 3  
let c = {  
  apple: a,  
  banana: b  
}  
let d = c.apple * 4
```

Output:



Explanation:

a variable that was inferred as a number was assigned a string value. TypeScript immediately showed a red squiggly line under the invalid code. This showed that TypeScript can detect type-related mistakes while writing the program.

b) Read the Type Error and try to understand what it means.

Explanation:

TypeScript shows an error saying that a string cannot be assigned to a number. This means that the value being given to the variable does not match its expected data type. The error message helps identify and correct such mistakes before running the program.